

JAVASCRIPT

SESSION 3

Math Object • Spread Operator • Conditional Control Flow

HANDWRITTEN STUDY NOTES

Simple language • best code examples • explanations • practice • answers

Built from the three Session 3 reference files + verified JS documentation

TABLE OF CONTENTS

Every topic is a separate handwritten page. Click a topic to jump there.

03	Session roadmap	30	Q7 - Divisible by 5 and 10
04	Math Object	31	Q8 - Student grading
05	Math.random()	32	Q9 - Age category
06	Random integer patterns	33	Q10 - Range check
07	Math.floor()	34	Q11 - Username + password
08	Math.sqrt()	35	Q12 - Restricted access
09	Math.pow()	36	Q13 - Positive and even
10	Math.min()	37	Q14 - Negative or odd
11	Math.max()	38	Q15 - Temperature classification
12	Math quick table	39	Q16 - Ticket type
13	Spread operator (...)	40	Q17 - Admin panel
14	Spread in function calls	41	Q18 - Passed all subjects
15	Spread + Math.max/min	42	Q19 - Positive / negative / zero
16	Spread in arrays + copying	43	Q20 - ATM simulation
17	Conditional statements	44	Additional practice Q21-Q25
18	if statement	45	Additional practice Q26-Q30
19	if / else	46	Challenge questions
20	else if chains	47	Answer key - source practice
21	Nested conditions	48	Answer key - additional + challenges
22	Branch order	49	Interview revision
23	Ternary operator	50	Quick revision sheet
24	if / else vs ternary	51	Mini project - Number Analyzer
25	Common decision patterns	52	Clean session code - Part 1
26	Q3 - Voting + validation	53	Clean session code - Part 2
27	Q4 - Larger of two	54	Common mistakes + final checklist
28	Q5 - Largest of three	55	Reference note
29	Q6 - Divisible by 3 and 5		

03 Session roadmap

Math tools -> spread values -> make decisions -> solve practical problems.

1. MATH OBJECT

A ready-made toolbox for common calculations.

2. SPREAD (...)

Open an array and place its items one-by-one where JavaScript expects separate values.

3. CONDITIONALS

Ask a true/false question and run the branch that matches.

The reference lesson follows this same sequence and then turns the ideas into validation, grading, access, range and simulation problems.

Study habit

Before writing code, say the rule in plain English. Example: 'The person needs age 18+ AND a valid ID.' Then translate each part into a condition.

04 Math Object

A built-in toolbox for mathematical constants and static methods.

Think of Math like a calculator shelf that already exists inside JavaScript. You call the methods directly.

Common forms

```
Math.random();  
Math.floor(8.9);  
Math.sqrt(81);  
Math.pow(2, 5);  
Math.min(4, 9, 1);  
Math.max(4, 9, 1);
```

Remember

Math is not something you create with new. Its methods are called directly on Math.

MDN confirms Math is a namespace-style built-in object whose properties and methods are static.

05 *Math.random()*

Returns a pseudo-random floating-point value from 0 inclusive to 1 exclusive.

```
const value = Math.random();  
console.log(value);
```

Memory rule

$0 \leq \text{value} < 1$

You may see 0.14, 0.73 or 0.999... The exact value changes between runs.

Security note

Math.random() is not for cryptographic security. Use a Web Crypto API when security-grade randomness is needed.

06 Random integer patterns

Turn `Math.random()` into the ranges you actually need.

```
// 0..9
const digit = Math.floor(Math.random() * 10);

// 1..6 (dice)
const dice = Math.floor(Math.random() * 6) + 1;

// 1..100
const score = Math.floor(Math.random() * 100) + 1;
```

1..6 step-by-step

random() -> $0 \leq x < 1$

$x * 6$ -> $0 \leq x < 6$

floor -> 0,1,2,3,4,5

+1 -> 1,2,3,4,5,6

07 *Math.floor()*

Always rounds down toward the lower integer.

```
Math.floor(4.9); // 4  
Math.floor(4.1); // 4  
Math.floor(-4.1); // -5
```

Easy trap

Do not think 'floor = remove decimals'. For negative values, floor can move farther from zero.

Formally, `Math.floor(x)` returns the greatest integer that is less than or equal to `x`.

08 Math.sqrt()

Find the positive square root.

```
const root = Math.sqrt(144);  
console.log(root); // 12
```

Think it like this

What number multiplied by itself gives 144? $12 \times 12 = 144$, so the answer is 12.

Real use

```
// Distance between two points  
const d = Math.sqrt((x2-x1)**2 + (y2-y1)**2);
```


09 *Math.pow()*

Raise a base to an exponent.

```
Math.pow(2, 5); // 32  
  
// modern equivalent  
2 ** 5;         // 32
```

Read *Math.pow(2, 5)* as '2 to the power 5'.

Interview tip

*Math.pow(base, exponent) and ** both express exponentiation; ** is the shorter modern form.*

10 *Math.min()*

Return the smallest value from the arguments.

```
const smallest = Math.min(18, 4, 29, 11, 7);  
console.log(smallest); // 4
```

Important

Math.min() receives individual numeric arguments. An array needs to be expanded first.

11 *Math.max()*

Return the largest value from the arguments.

```
const largest = Math.max(18, 4, 29, 11, 7);  
console.log(largest); // 29
```

When the numbers already live inside an array, spread connects naturally with `Math.max()`.

12 Math quick table

Fast recall before solving questions.

Method	Meaning	Example
<code>Math.random()</code>	$0 \leq \text{value} < 1$	<code>Math.random()</code>
<code>Math.floor(x)</code>	round down	<code>Math.floor(4.9) -> 4</code>
<code>Math.sqrt(x)</code>	square root	<code>Math.sqrt(81) -> 9</code>
<code>Math.pow(a,b)</code>	a raised to b	<code>2 ** 5 -> 32</code>
<code>Math.min(...)</code>	smallest	<code>Math.min(8,2,5) -> 2</code>
<code>Math.max(...)</code>	largest	<code>Math.max(8,2,5) -> 8</code>

Memory line

random -> floor -> sqrt -> pow -> min/max

13 Spread operator (...)

Expand an iterable such as an array into individual items.

```
const marks = [72, 88, 91];  
console.log(...marks);  
// 72 88 91
```

Mental model

Read `...marks` as: *'take the items inside marks and put them here one-by-one.'*

MDN: spread syntax can expand iterable values in function calls and array literals; object spread has its own behavior.

14 Spread in function calls

Why spread helps when a function expects separate arguments.

```
function total(a, b, c) {  
  return a + b + c;  
}  
  
const values = [10, 20, 30];  
console.log(total(...values)); // 60
```

Without spread, total(values) passes one array. With spread, JavaScript effectively receives total(10, 20, 30).

15 Spread + Math.max/min

The key Session 3 connection.

```
const numbers = [10, 44, 2, 7, 31, 5];  
  
const highest = Math.max(...numbers);  
const lowest = Math.min(...numbers);  
  
console.log(highest); // 44  
console.log(lowest);  // 2
```

Why it works

Math.max(10, 44, 2, ...) is valid. Math.max(numbers) is not the same call. Spread turns the array into separate arguments.

16 Spread in arrays + copying

A useful extension: combine arrays and make shallow copies.

```
const first = [1, 2, 3];  
const combined = [0, ...first, 4];  
// [0,1,2,3,4]
```

```
const copy = [...first];  
copy.push(99);
```

Shallow copy

The new array is separate at the outer level. Nested objects/arrays are still shared references.

17 Conditional statements

A condition is a question that becomes true or false.

```
if (condition) {  
  // true branch  
} else if (anotherCondition) {  
  // next branch  
} else {  
  // fallback branch  
}
```

Flow

Check condition -> choose one branch -> run it -> continue.

The reference lesson defines conditional statements as control-flow constructs that choose different blocks based on a boolean condition.

18 if statement

Run a block only when the condition is true.

```
const age = 21;

if (age >= 18) {
  console.log("Adult");
}
```

Read the question

age >= 18 asks: 'Is age at least 18?' Since 21 satisfies the condition, the block runs.

19 if / else

Two possible paths.

```
const age = 16;

if (age >= 18) {
  console.log("Can vote");
} else {
  console.log("Cannot vote");
}
```

Exactly one branch runs. If the condition is false, control jumps to else.

20 *else if chains*

Use multiple branches for ranges or categories.

```
const marks = 78;

if (marks >= 90) {
  console.log("A+");
} else if (marks >= 75) {
  console.log("A");
} else if (marks >= 40) {
  console.log("Pass");
} else {
  console.log("Fail");
}
```

Order matters

JavaScript checks from top to bottom and stops at the first matching branch. Put stronger/higher rules first.

21 Nested conditions

An if can contain another if.

```
const age = 20;
const hasID = true;

if (age >= 18) {
  if (hasID) {
    console.log("Access granted");
  }
}
```

Cleaner when the rule is simple

If both checks must pass, combine them with && to make the rule easier to read.

Cleaner version

```
if (age >= 18 && hasID) {
  console.log("Access granted");
}
```

22 Branch order

The first true branch wins.

```
const temp = 40;

if (temp > 35) {
  console.log("Hot");
} else if (temp >= 15) {
  console.log("Normal");
} else {
  console.log("Cold");
}
```

At 40, the first condition is true, so JavaScript never evaluates the later branches for execution.

Remember

Branch order is part of the logic, not just formatting.

23 Ternary operator

A compact two-way conditional that produces a value.

```
const age = 19;

const status = age >= 18
  ? "Adult"
  : "Minor";

console.log(status); // Adult
```

Pattern

condition ? valueWhenTrue : valueWhenFalse

It is called ternary because it has three operands: the condition, the true result and the false result.

24 *if / else vs ternary*

Same decision idea, different style.

Style	Best fit
<i>if / else</i>	<i>Run blocks, perform several steps, or handle multiple branches.</i>
<i>ternary</i>	<i>Choose one value/message from two alternatives.</i>

```
// if / else
let result;
if (age >= 18) result = "Eligible";
else result = "Not eligible";

// ternary
const result2 = age >= 18 ? "Eligible" : "Not eligible";
```

Practical rule

Keep ternaries short. If the expression becomes hard to scan, use if/else.

25 Common decision patterns

Turn plain-English rules into code.

```
// Even
n % 2 === 0

// Inside a range
n >= 10 && n <= 50

// BOTH requirements
age >= 18 && hasID

// EITHER requirement
isAdmin || isOwner

// Positive / negative / zero
n > 0
n < 0
else -> zero
```

Best habit

Write the English rule first. Then convert each part into a comparison. Finally connect the parts with && or ||.

26 Q3 - Voting + validation

Practical exercise rewritten from the Session 3 reference set.

```
const age = 28;

if (age < 0) {
  console.log("Age must be positive");
} else if (age >= 18) {
  console.log("Eligible to vote");
} else {
  console.log("Not eligible to vote");
}
```

How it works

Validation comes first, so invalid values are rejected before eligibility is decided.

Expected result

Eligible to vote

27 Q4 - Larger of two

Practical exercise rewritten from the Session 3 reference set.

```
const a = 23;  
const b = 34;  
const larger = a > b ? a : b;  
console.log(larger);
```

How it works

The ternary selects a only when a is bigger. Otherwise b is used.

Expected result

34

28 Q5 - Largest of three

Practical exercise rewritten from the Session 3 reference set.

```
const a = 23;  
const b = 34;  
const c = 56;  
const largest = Math.max(a, b, c);  
console.log(largest);
```

How it works

Math.max compares the supplied arguments and returns the highest one.

Expected result

56

29 Q6 - Divisible by 3 and 5

Practical exercise rewritten from the Session 3 reference set.

```
const n = 15;

if (n % 3 === 0 && n % 5 === 0) {
  console.log("Both");
} else if (n % 3 === 0 || n % 5 === 0) {
  console.log("Only one");
} else {
  console.log("Neither");
}
```

How it works

Remainder 0 means divisible. && checks both rules; || checks whether at least one rule is true.

Expected result

Both

30 Q7 - Divisible by 5 and 10

Practical exercise rewritten from the Session 3 reference set.

```
const n = 15;

if (n % 5 === 0 && n % 10 === 0) {
  console.log("Both");
} else if (n % 5 === 0 || n % 10 === 0) {
  console.log("Only one");
} else {
  console.log("Neither");
}
```

How it works

15 is divisible by 5, but not by 10, so the 'Only one' branch runs.

Expected result

Only one

31 Q8 - Student grading

Practical exercise rewritten from the Session 3 reference set.

```
const marks = 99;
let grade = "";

if (marks >= 0 && marks <= 100) {
  if (marks >= 90) grade = "A+";
  else if (marks >= 70) grade = "A";
  else if (marks >= 40) grade = "B";
  else if (marks >= 30) grade = "C";
  else grade = "Fail";
} else {
  grade = "Invalid marks";
}

console.log(grade);
```

How it works

First validate the allowed range. Then the nested grade ladder chooses the first matching band.

Expected result

A+

32 Q9 - Age category

Practical exercise rewritten from the Session 3 reference set.

```
const age = 16;

if (age <= 0) {
  console.log("Invalid age");
} else if (age < 12) {
  console.log("Child");
} else if (age <= 18) {
  console.log("Teenager");
} else {
  console.log("Adult");
}
```

How it works

The age is checked from top to bottom. The first matching category wins.

Expected result

Teenager

33 Q10 - Range check

Practical exercise rewritten from the Session 3 reference set.

```
const n = 34;

if (n >= 10 && n <= 50) {
  console.log("Inside range");
} else {
  console.log("Outside range");
}
```

How it works

The number must satisfy both boundary conditions because the range is inclusive.

Expected result

Inside range

34 Q11 - Username + password

Practical exercise rewritten from the Session 3 reference set.

```
const username = "admin_user";
const password = "SecurePassword123";

if (
  username === "admin_user" &&
  password === "SecurePassword123"
) {
  console.log("Login accepted");
} else {
  console.log("Login rejected");
}
```

How it works

Use strict equality for both fields and && because both credentials must match.

Expected result

Login accepted

35 Q12 - Restricted access

Practical exercise rewritten from the Session 3 reference set.

```
const age = 20;  
const hasID = true;  
  
if (age >= 18 && hasID) {  
  console.log("Access granted");  
} else {  
  console.log("Access denied");  
}
```

How it works

The age test and the ID check are both required, so && is the natural operator.

Expected result

Access granted

36 Q13 - Positive and even

Practical exercise rewritten from the Session 3 reference set.

```
const n = 14;

if (n > 0 && n % 2 === 0) {
  console.log("Positive and even");
} else {
  console.log("Not both");
}
```

How it works

One condition checks sign; the other checks whether division by 2 leaves remainder 0.

Expected result

Positive and even

37 Q14 - Negative or odd

Practical exercise rewritten from the Session 3 reference set.

```
const n = -7;

if (n < 0 || n % 2 !== 0) {
  console.log("Negative or odd");
} else {
  console.log("Neither");
}
```

How it works

OR needs only one true side. -7 is already negative, so the condition succeeds.

Expected result

Negative or odd

38 Q15 - Temperature classification

Practical exercise rewritten from the Session 3 reference set.

```
const temp = 28;

if (temp > 35) {
  console.log("Hot");
} else if (temp >= 15) {
  console.log("Normal");
} else {
  console.log("Cold");
}
```

How it works

The code checks the hottest band first, then normal, then the final cold fallback.

Expected result

Normal

39 Q16 - Ticket type

Practical exercise rewritten from the Session 3 reference set.

```
const age = 10;

const ticket = age < 12
  ? "Child Ticket"
  : "Adult Ticket";

console.log(ticket);
```

How it works

A short two-way decision is a good fit for ternary.

Expected result

Child Ticket

40 Q17 - Admin panel

Practical exercise rewritten from the Session 3 reference set.

```
const isLoggedIn = true;
const isAdmin = true;

if (isLoggedIn && isAdmin) {
  console.log("Admin access");
} else {
  console.log("Access denied");
}
```

How it works

Both Boolean requirements must be true. If either is false, the user is rejected.

Expected result

Admin access

41 Q18 - Passed all subjects

Practical exercise rewritten from the Session 3 reference set.

```
const math = 45;
const science = 38;
const english = 50;
const pass = 35;

if (math >= pass &&
    science >= pass &&
    english >= pass) {
  console.log("Passed all subjects");
} else {
  console.log("Failed one or more subjects");
}
```

How it works

All three comparisons must succeed. One failed subject is enough to enter else.

Expected result

Passed all subjects

42 Q19 - Positive / negative / zero

Practical exercise rewritten from the Session 3 reference set.

```
const n = 0;

if (n > 0) {
  console.log("Positive");
} else if (n < 0) {
  console.log("Negative");
} else {
  console.log("Zero");
}
```

How it works

Zero is neither positive nor negative, so the final else naturally becomes the zero case.

Expected result

Zero

43 Q20 - ATM simulation

Practical exercise rewritten from the Session 3 reference set.

```
const correctPin = 4567;
const enteredPin = 4567;
let balance = 50000;
const withdrawal = 10000;

if (enteredPin === correctPin) {
  if (withdrawal > 0 && withdrawal <= balance) {
    balance -= withdrawal;
    console.log("Withdrawal successful");
    console.log("Balance:", balance);
  } else if (withdrawal > balance) {
    console.log("Insufficient balance");
  } else {
    console.log("Invalid amount");
  }
} else {
  console.log("Wrong PIN");
}
```

How it works

Authentication is checked first. Only a correct PIN lets the withdrawal rules run.

Expected result

Withdrawal successful / Balance: 40000

44 Additional practice Q21-Q25

Use the same building blocks with fresh values.

```
// Q21 - dice
const dice = Math.floor(Math.random() * 6) + 1;

// Q22 - square root
const root = Math.sqrt(81);

// Q23 - largest array value
const values = [12, 48, 7, 19, 33];
const largest = Math.max(...values);

// Q24 - age category
const age = 16;
if (age < 12) console.log("Child");
else if (age <= 18) console.log("Teenager");
else console.log("Adult");

// Q25 - eligible or not
const result = 21 >= 18 ? "Eligible" : "Not eligible";
```

Answers

Q21 -> 1..6, Q22 -> 9, Q23 -> 48, Q24 -> Teenager, Q25 -> Eligible

45 Additional practice Q26-Q30

More spread, range and decision patterns.

```
// Q26
const values = [23, 4, 18, 11, 9];
const smallest = Math.min(...values);

// Q27
const n = 37;
if (n >= 20 && n <= 40) console.log("Inside");

// Q28
const power = Math.pow(3, 4);

// Q29
const age = 22, hasID = true;
if (age >= 18 && hasID) console.log("Access granted");

// Q30
const temp = 12;
if (temp > 35) console.log("Hot");
else if (temp >= 15) console.log("Normal");
else console.log("Cold");
```

Answers

Q26 -> 4, Q27 -> Inside, Q28 -> 81, Q29 -> Access granted, Q30 -> Cold

46 Challenge questions

Predict first; then verify by running the code.

1) What integers can `Math.floor(Math.random() * 10)` produce?

Answer

0 through 9

2) What reaches `Math.max(...[8, 2, 17, 5])` and what is the result?

Answer

8, 2, 17, 5 are individual arguments; result = 17

3) Marks = 72 in the grading ladder. Which grade matches?

Answer

A

4) `age = 20` and `hasID = false`. What happens with `age >= 18 && hasID`?

Answer

Access denied

5) `temp = 40`. Which branch wins?

Answer

Hot

47 Answer key - source practice

Quick self-check for Q3-Q20.

Q	Result
Q3	<i>Eligible to vote</i>
Q4	<i>34</i>
Q5	<i>56</i>
Q6	<i>Divisible by both 3 and 5</i>
Q7	<i>Divisible by only one of 5 or 10</i>
Q8	<i>A+</i>
Q9	<i>Teenager</i>
Q10	<i>34 is inside 10..50</i>
Q11	<i>Login accepted</i>
Q12	<i>Access granted</i>
Q13	<i>14 is positive and even</i>
Q14	<i>-7 is negative or odd</i>
Q15	<i>Normal</i>
Q16	<i>Child Ticket</i>
Q17	<i>Admin access</i>
Q18	<i>Passed all subjects</i>
Q19	<i>Zero</i>
Q20	<i>With correct PIN: withdrawal succeeds, balance 40000</i>

48 Answer key - additional + challenges

Keep the reason attached to the answer.

```
Q21 -> 1..6
Q22 -> 9
Q23 -> 48
Q24 -> Teenager
Q25 -> Eligible

Q26 -> 4
Q27 -> Inside
Q28 -> 81
Q29 -> Access granted
Q30 -> Cold
```

Challenges:

```
1 -> 0..9
2 -> 17
3 -> A
4 -> Access denied
5 -> Hot
```

Why this matters

Each answer points back to a reusable rule: random range, floor, spread, branch order, comparison and logical AND.

49 Interview revision

Short answers for a viva or interview.

What is the Math object?

A built-in namespace object that provides mathematical constants and static methods.

What does Math.random() return?

A pseudo-random floating-point number from 0 inclusive to 1 exclusive.

What does Math.floor() do?

It returns the greatest integer less than or equal to the input - in simple words, it rounds down.

What does spread do?

It expands iterable values, such as array items, into argument positions or array elements.

Why use ...array with Math.max()?

Because Math.max expects separate numeric arguments, while an array is one value.

What is a conditional statement?

A control-flow structure that chooses which block should run based on a condition.

What is ternary?

A three-part expression: condition ? trueValue : falseValue.

50 Quick revision sheet

One-page cheat sheet.

Concept	Quick memory
<code>Math.random()</code>	$0 \leq \text{value} < 1$
<code>Math.floor(x)</code>	round down
<code>Math.sqrt(x)</code>	square root
<code>Math.pow(a,b)</code>	a raised to b
<code>Math.min(...)</code>	smallest argument
<code>Math.max(...)</code>	largest argument
<code>Spread (...)</code>	expand iterable items
<code>if / else</code>	choose a branch
<code>else if</code>	try another condition
<code>ternary</code>	condition ? true : false
<code>&&</code>	both must be true
<code> </code>	at least one must be true
<code>% === 0</code>	common divisibility check
<code>n >= min && n <= max</code>	inclusive range

Memory chain

Math -> Spread -> Conditions -> Practice

51 Mini project - Number Analyzer

Combine Math, spread and conditions in one realistic exercise.

```
const numbers = [12, 7, 34, 18, 5];

const highest = Math.max(...numbers);
const lowest = Math.min(...numbers);

let total = 0;
for (const n of numbers) {
  total += n;
}

const average = total / numbers.length;

console.log("Highest:", highest);
console.log("Lowest:", lowest);
console.log("Average:", average);

for (const n of numbers) {
  if (n > 20) console.log(n, "-> high");
  else if (n >= 10) console.log(n, "-> medium");
  else console.log(n, "-> low");
}
```

What this project uses

Spread + Math.max/min + arithmetic + if/else-if/else. It is a good bridge from isolated syntax to actual problem solving.

52 Clean session code - Part 1

Main Math + spread patterns in one place.

```
console.log("=== SESSION 3 ===");

const randomValue = Math.random();
const dice = Math.floor(Math.random() * 6) + 1;

const root = Math.sqrt(144);
const power = Math.pow(2, 5);

const highest = Math.max(8, 21, 4, 17);
const lowest = Math.min(8, 21, 4, 17);

const values = [10, 20, 30, 40];
const arrayHighest = Math.max(...values);
const arrayLowest = Math.min(...values);

console.log({
  randomValue,
  dice,
  root,
  power,
  highest,
  lowest,
  arrayHighest,
  arrayLowest
});
```

53 Clean session code - Part 2

Decision-flow practice in a compact program.

```
const age = 20;
const hasID = true;
const marks = 82;

if (age >= 18 && hasID) {
  console.log("Access granted");
} else {
  console.log("Access denied");
}

let grade;
if (marks >= 90) grade = "A+";
else if (marks >= 70) grade = "A";
else if (marks >= 40) grade = "B";
else grade = "Fail";

console.log("Grade:", grade);

const ticket = age < 12 ? "Child Ticket" : "Adult Ticket";
console.log("Ticket:", ticket);
```

54 Common mistakes + final checklist

Use this before moving to Session 4.

- *Math.random()* never returns exactly 1. - *Math.floor(-4.1)* is -5. - *Math.max(numbers)* is not the same as *Math.max(...numbers)*. - Spread expands; rest collects. - Avoid deeply nested ternaries. - else-if branches are checked top to bottom. - BOTH -> think &&. EITHER -> think ||. - Validate input before applying business rules. - Write the rule in plain English before writing the condition.

Final self-test

Can you explain *Math.random()*, *Math.floor()*, *Math.sqrt()*, *Math.pow()*, *Math.min()*, *Math.max()*, spread, if/else-if/else, ternary, &&, || and range checks without looking?

55 Reference note

What was used to build these notes.

The uploaded JavaScript file supplies the Session 3 concepts and the Q3-Q20 problem set; it covers Math methods, spread with Math, conditionals, ternary and practical decision problems.

The uploaded HTML file shows the browser page loading the Session 3 JavaScript using a script tag.

The uploaded Session 3 PDF supplies the lesson order, additional practice Q21-Q30, challenge questions, answer keys, interview revision and quick-revision structure.

The uploaded diagram was used as visual inspiration for the three main branches: Math Object, Spread Operator and Conditional Statements.

External verification

MDN and W3Schools were checked for Math.random() ranges, Math.floor(), Math.min()/max(), spread syntax and conditional/ternary behavior.

This notebook rewrites the material in simpler language and uses fresh example values/code while preserving the same learning objectives.